FILMORA

BACKEND ARCHITECTURE & REQUIREMENTS SPECIFICATION

Project Name: FILMORA
Project Type: Free Online Movie Streaming Platform
Document Type: Backend Technical Specification
Version: 1.0
Status: Proposed Design
Backend Language: JavaScript
Runtime: Node.js
Framework: Express.js (proposed)
Database: PostgreSQL (proposed)
API Architecture: REST API (proposed)

TABLE OF CONTENTS

1. Introduction
2. Backend System Objectives
3. Backend Technology Stack
4. Backend Architecture
5. Backend Folder Structure
6. Authentication and Authorization
7. User Management
8. Movie Management
9. Genre and Category Management
10. Cast and Crew Management
11. Media Management
12. Video Streaming Management
13. Subtitle and Audio Management
14. Watch History Management
15. Watchlist and Favorites Management
16. Reviews and Ratings Management
17. Notification Management
18. Administration Management

19. API Architecture
20. Middleware Requirements
21. Validation and Error Handling
22. Security Requirements
23. Performance and Optimization
24. Logging and Monitoring
25. Testing Requirements
26. Deployment Requirements
27. Backup and Recovery
28. Backend Development Phases

---

1. INTRODUCTION

The FILMORA backend is the server-side system responsible for managing the operations of the movie streaming platform.

It connects the frontend interface to the database, handles user authentication, manages movie information, controls streaming access and processes user activities.

The backend will serve as the central operational system of FILMORA.

1.1 Main responsibilities

- Registering and authenticating users.
- Managing user accounts.
- Managing movies and movie categories.
- Processing search requests.
- Managing movie posters and media.
- Authorizing full-length movie streaming.
- Managing subtitles and audio tracks.
- Tracking viewing progress.
- Managing watchlists and favorites.
- Processing movie reviews and ratings.
- Managing administrative operations.

- Protecting platform resources.

2. BACKEND SYSTEM OBJECTIVES

The backend must satisfy the following objectives:

1. Provide reliable communication between frontend and database.
2. Protect user accounts and private information.
3. Support authenticated movie streaming.
4. Provide organized movie management.
5. Support multiple concurrent users.
6. Provide secure administrative functionality.
7. Maintain consistent database records.
8. Support future system expansion.
9. Provide reliable error handling.
10. Maintain acceptable response times.

3. BACKEND TECHNOLOGY STACK

The proposed technology stack is:

Component	**Technology**	**Purpose**
Programming language	JavaScript	Backend development
Runtime	Node.js	Server execution
Framework	Express.js	HTTP server and routing
Database	PostgreSQL	Structured data storage
ORM	Prisma	Database communication
Authentication	Secure sessions	User authentication
Password hashing	Argon2id	Password protection
Validation	Zod	Input validation
API architecture	REST API	Frontend communication
Video processing	FFmpeg	Video preparation
Streaming format	HLS	Video delivery

Component	Technology	Purpose
Testing	Jest or Vitest	Backend testing
Logging	Pino	Application logging

4. BACKEND ARCHITECTURE

The backend will use a modular architecture.

The frontend will communicate with the backend through HTTP requests.

The backend will process requests, validate data, check permissions, communicate with PostgreSQL and return responses.

4.1 System components

A. Frontend

Responsible for:

- Displaying website pages.
- Sending user requests.
- Displaying movie information.
- Playing authorized video content.
- Displaying account information.

B. API layer

Responsible for:

- Receiving HTTP requests.
- Routing requests.
- Returning HTTP responses.
- Applying middleware.

C. Controller layer

Responsible for:

- Receiving validated requests.
- Calling business logic.
- Returning appropriate responses.

D. Service layer

Responsible for:

- Authentication operations.
- Movie operations.
- Streaming authorization.
- User activities.
- Administrative operations.

E. Database layer

Responsible for:

- Storing user information.
- Storing movie information.
- Maintaining relationships.
- Managing viewing history.
- Managing reviews and ratings.

F. Media storage

Responsible for storing:

- Full-length movie files.
- Movie posters.
- Thumbnails.
- Backdrops.
- Subtitle files.
- Streaming manifests and segments.

5. BACKEND FOLDER STRUCTURE

The proposed project structure is:

FILMORA/

backend/

- src/
    - config/
    - controllers/
    - services/
    - routes/

    - middleware/
    - validators/
    - repositories/
    - utils/
    - jobs/
    - streaming/
    - app.js
    - server.js
- prisma/
    - schema.prisma
    - migrations/
- tests/
- package.json
- .env
- .env.example

frontend/

- index.html
- css/
- js/
- assets/

README.md

5.1 Folder responsibilities

Folder	Responsibility
config	Application configuration
controllers	Request handling
services	Business logic
routes	API endpoint definitions

Folder	Responsibility
middleware	Authentication and request processing
validators	Input validation
repositories	Database access
utils	Reusable helper functions
jobs	Background processing
streaming	Video authorization and delivery
tests	Automated testing

6. AUTHENTICATION AND AUTHORIZATION MODULE

This module manages user registration, login and access permissions.

6.1 Registration functionality

The backend must:

- Receive registration information.
- Validate submitted information.
- Check username uniqueness.
- Check email uniqueness.
- Hash the password.
- Create the user account.
- Create the user profile.
- Assign the default user role.
- Initiate email verification.

6.2 Login functionality

The backend must:

- Receive email and password.
- Locate the account.
- Verify the password.
- Check account status.
- Create a secure authenticated session.

- Return an appropriate response.

6.3 Additional authentication functionality

- Logout.
- Password recovery.
- Password changes.
- Email verification.
- Session expiration.
- Session revocation.

6.4 Authorization roles

Initial system roles:

Role	Permissions
Administrator	Complete platform management
Content Manager	Movie and media management
Moderator	User-generated content moderation
Registered User	Movie viewing and personal account features

7. USER MANAGEMENT MODULE

Responsible for managing registered accounts.

Functions:

- Retrieve user information.
- Update profiles.
- Change passwords.
- Update profile pictures.
- Manage account status.
- Manage user preferences.
- Manage active sessions.
- Delete or deactivate accounts.

8. MOVIE MANAGEMENT MODULE

Responsible for managing movie information.

Functions:

- Add movies.
- Edit movie details.
- Delete or archive movies.
- Publish movies.
- Unpublish movies.
- Retrieve movie details.
- Retrieve featured movies.
- Retrieve recently added movies.
- Retrieve popular movies.
- Retrieve recommended movies.

Movie information includes:

- Movie title.
- Description.
- Release date.
- Duration.
- Language.
- Age classification.
- Genre.
- Poster.
- Backdrop.
- Cast.
- Director.
- Streaming availability.

Only authorized content may be published.

9. GENRE AND CATEGORY MANAGEMENT MODULE

Responsible for movie classification.

Functions:

- Create genres.

- Edit genres.
- Remove genres.
- Assign multiple genres to movies.
- Create website categories.
- Organize featured collections.
- Manage trending collections.

10. CAST AND CREW MANAGEMENT MODULE

Responsible for managing film professionals.

Functions:

- Add actors.
- Add directors.
- Add producers.
- Edit professional information.
- Assign people to movies.
- Display movie credits.

11. MEDIA MANAGEMENT MODULE

Responsible for managing uploaded images and other media.

Functions:

- Upload movie posters.
- Upload thumbnails.
- Upload backdrops.
- Validate file types.
- Validate file sizes.
- Store media files.
- Generate image metadata.
- Update media references.
- Remove unused media files.

Media storage will be external to PostgreSQL.

12. VIDEO STREAMING MANAGEMENT MODULE

This is one of the most important backend modules.

It manages authorized full-length movie streaming.

Functions:

- Register video assets.
- Validate uploaded video files.
- Process video files.
- Generate streaming versions.
- Generate HLS manifests.
- Generate video segments.
- Manage different resolutions.
- Authorize viewing sessions.
- Track playback activity.
- Control streaming access.

12.1 Supported video qualities

Proposed qualities:

- 360p.
- 480p.
- 720p.
- 1080p.

Actual availability will depend on the source video.

12.2 Streaming requirements

The system must support:

- Adaptive bitrate streaming.
- Playback authorization.
- Secure media delivery.
- Resume watching.
- Video progress tracking.
- Playback error handling.
- Multiple concurrent viewers.

The backend will issue authorized streaming access. Video delivery itself will be handled by appropriate storage and CDN infrastructure.

13. SUBTITLE AND AUDIO MANAGEMENT MODULE

Functions:

- Upload subtitles.
- Associate subtitles with movies.
- Manage subtitle languages.
- Manage audio tracks.
- Set default subtitle language.
- Set default audio language.
- Validate subtitle formats.

14. WATCH HISTORY MODULE

Responsible for tracking user viewing activities.

Functions:

- Record movie viewing sessions.
- Save playback position.
- Update viewing progress.
- Identify completed movies.
- Retrieve recent viewing history.
- Resume movies from saved positions.

15. WATCHLIST AND FAVORITES MODULE

Functions:

- Create watchlists.
- Add movies to watchlists.
- Remove movies.
- Retrieve saved movies.
- Add favorites.
- Remove favorites.
- Display favorite collections.

16. REVIEWS AND RATINGS MODULE

Functions:

- Submit movie ratings.
- Submit movie reviews.
- Edit permitted reviews.
- Delete permitted reviews.
- Retrieve movie reviews.
- Calculate average ratings.
- Moderate inappropriate content.

17. NOTIFICATION MODULE

Responsible for communicating important information to users.

Functions:

- Create notifications.
- Retrieve notifications.
- Mark notifications as read.
- Send account-related notifications.
- Send platform announcements.
- Manage notification preferences.

18. ADMINISTRATION MODULE

Provides backend functionality for the FILMORA administration dashboard.

Functions:

- Manage users.
- Manage administrator roles.
- Manage movies.
- Manage genres.
- Manage categories.
- Manage media uploads.
- Manage streaming content.
- Review content reports.

- Manage platform settings.
- View platform statistics.
- Review administrative logs.

19. API ARCHITECTURE

FILMORA will use REST API endpoints.

19.1 Authentication endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register account
POST	/api/auth/login	Authenticate user
POST	/api/auth/logout	End session
POST	/api/auth/forgot-password	Request password reset
POST	/api/auth/reset-password	Reset password
POST	/api/auth/verify-email	Verify email

19.2 User endpoints

Method	Endpoint	Description
GET	/api/users/me	Retrieve current user
PUT	/api/users/me	Update profile
PUT	/api/users/me/password	Change password
DELETE	/api/users/me	Request account deletion

19.3 Movie endpoints

Method	Endpoint	Description
GET	/api/movies	Retrieve movies
GET	/api/movies/:id	Retrieve movie details
GET	/api/movies/featured	Featured movies
GET	/api/movies/popular	Popular movies
GET	/api/movies/recent	Recently added movies

Method	Endpoint	Description
GET	/api/genres	Retrieve genres
GET	/api/categories	Retrieve categories

19.4 Streaming endpoints

Method	Endpoint	Description
POST	/api/stream/:movieId/session	Request streaming authorization
GET	/api/stream/:movieId/manifest	Obtain authorized manifest access
POST	/api/stream/progress	Save viewing progress
POST	/api/stream/session/end	End streaming session

19.5 Watchlist endpoints

Method	Endpoint	Description
GET	/api/watchlist	Retrieve watchlist
POST	/api/watchlist	Add movie
DELETE	/api/watchlist/:movieId	Remove movie
GET	/api/favorites	Retrieve favorites
POST	/api/favorites	Add favorite

19.6 Administration endpoints

Method	Endpoint	Description
GET	/api/admin/dashboard	Dashboard statistics
GET	/api/admin/users	Manage users
POST	/api/admin/movies	Create movie
PUT	/api/admin/movies/:id	Update movie
DELETE	/api/admin/movies/:id	Archive movie
POST	/api/admin/media/upload	Upload authorized media

All endpoints must implement appropriate access controls and request validation.

20. MIDDLEWARE REQUIREMENTS

The backend will include:

1. Authentication middleware.
2. Authorization middleware.
3. Request validation middleware.
4. Error handling middleware.
5. Rate limiting middleware.
6. Logging middleware.
7. CORS configuration.
8. Secure HTTP header configuration.
9. Request size restrictions.

21. VALIDATION AND ERROR HANDLING

The backend must validate:

- Email formats.
- Username requirements.
- Password requirements.
- Movie information.
- Image uploads.
- Video uploads.
- User reviews.
- API request parameters.

The backend must return consistent error responses.

Examples:

- 400 – Bad Request.
- 401 – Unauthorized.
- 403 – Forbidden.
- 404 – Not Found.
- 409 – Conflict.
- 422 – Validation Error.
- 429 – Too Many Requests.

- 500 – Internal Server Error.

Sensitive system information must not be exposed in public error responses.

22. SECURITY REQUIREMENTS

FILMORA must implement:

- Argon2id password hashing.
- Secure session management.
- HTTP-only secure cookies.
- CSRF protection where applicable.
- SQL injection protection.
- Input validation.
- Role-based authorization.
- Login rate limiting.
- Restricted administrator operations.
- Secure environment variables.
- Private video storage.
- Authorized streaming access.
- HTTPS communication.

23. PERFORMANCE AND OPTIMIZATION

The backend should support:

- Database connection pooling.
- Efficient database queries.
- Pagination.
- Caching of frequently requested information.
- Image optimization.
- Background video processing.
- CDN integration.
- Monitoring of slow requests.

24. LOGGING AND MONITORING

The system must record:

- Application errors.
- Authentication events.
- Administrative activities.
- Video processing failures.
- Database connection failures.
- API performance information.
- Security-related events.

Passwords, session secrets and private credentials must never appear in logs.

25. TESTING REQUIREMENTS

The backend must undergo:

- Authentication testing.
- Registration testing.
- Database integration testing.
- API endpoint testing.
- Permission testing.
- Streaming authorization testing.
- File upload testing.
- Error handling testing.
- Performance testing.
- Security testing.

26. DEPLOYMENT REQUIREMENTS

The backend deployment environment must provide:

- Node.js runtime.
- PostgreSQL connection.
- Environment variable configuration.
- HTTPS support.
- Secure file storage.
- CDN integration.
- Application logging.

- Database backups.
- Health monitoring.

27. BACKUP AND RECOVERY

The backend must support:

- Automated database backups.
- Database restoration.
- Media storage recovery.
- Application configuration backups.
- Disaster recovery procedures.

28. BACKEND DEVELOPMENT PHASES

Phase	Description
1	Initialize Node.js project
2	Configure Express.js
3	Connect PostgreSQL
4	Configure Prisma
5	Implement authentication
6	Implement user management
7	Implement movie management
8	Implement categories and genres
9	Implement media management
10	Implement video processing and streaming
11	Implement viewing history
12	Implement watchlists and favorites
13	Implement reviews and ratings
14	Implement notifications
15	Implement administration

Phase	Description
16	Security testing
17	Performance testing
18	Production deployment

FINAL BACKEND DESIGN PRINCIPLE

FILMORA will use a modular backend architecture built around Node.js and Express.js.

The backend will provide secure API communication, centralized business logic, database access, user authentication, administrative operations and authorized movie streaming.

The architecture will initially remain a modular monolith to reduce complexity and simplify maintenance while allowing future expansion.

DOCUMENT STATUS: PROPOSED — AWAITING APPROVAL

FILMORA — YOUR WORLD OF MOVIES.